SVM-Gateway: Audit Diff Doc

This document lists every program-source change between `audit-main` and `audit-main-fixes` for the SVM gateway program under `contracts/svm-gateway/programs/universal-gateway/src/`.

Comparison used:

```
git diff origin/audit-main..audit-main-fixes -- contracts/svm-gateway/programs/universal-gateway/src/
```

errors.rs

1. `GatewayError`: added a dedicated gas budget underflow error

Finalize gas settlement now reimburses actual Solana costs instead of blindly paying the full signed gas amount. The program needed a dedicated error to reject finalizes whose signed `gas_fee` cannot cover the computed minimum `gas_used`.

Before:

```
pub enum GatewayError {
    #[msg("Fee vault has insufficient balance to reimburse relayer")]
    InsufficientFeePool,
}
```

After:

```
pub enum GatewayError {
    #[msg("Fee vault has insufficient balance to reimburse relayer")]
    InsufficientFeePool,
```

```
    #[msg("gas_fee is below the minimum required gas_used for
this finalize")]
    InsufficientGasBudget,
}
```

Added `InsufficientGasBudget` so underfunded finalizes fail explicitly during gas settlement.

instructions/admin.rs

1. `AdminAction` : removed the global paused guard from admin-only config setters

The admin needs to keep emergency recovery knobs usable while the gateway is paused. Blocking all admin-only config instructions behind `!config.paused` prevented fee shutdowns and config corrections during incidents.

Before:

```
#[derive(Accounts)]
pub struct AdminAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = !config.paused @ GatewayError::Paused,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,

    pub admin: Signer<'info>,
}
```

After:

```
#[derive(Accounts)]
pub struct AdminAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,

    pub admin: Signer<'info>,
}
```

Admin-only config instructions are now callable even while the gateway is paused.

2. `ProposeAuthoritiesAction`, `AcceptAdminAction`, `AcceptPauserAction`: replaced one-step authority overwrites with a two-step accept flow

Immediate authority overwrites were risky because a typo or wrong destination key took effect instantly. The admin/pauser rotation path now stages pending authorities first, then requires the incoming authority to accept.

Before:

```
/// Authority update action (available while paused).
/// Updates admin and/or pauser in one instruction.
#[derive(Accounts)]
pub struct SetAuthoritiesAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
```

```
    )]
    pub config: Account<'info, Config>,

    pub admin: Signer<'info>,
}
```

After:

```
/// Authority update action (available while paused).
/// Proposes admin and/or pauser updates. Proposed authoritie
s must accept explicitly.
#[derive(Accounts)]
pub struct ProposeAuthoritiesAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,

    pub admin: Signer<'info>,
}

#[derive(Accounts)]
pub struct AcceptAdminAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.pending_admin == pending_admin.ke
y() @ GatewayError::Unauthorized
    )]
    pub config: Account<'info, Config>,
```

```
    pub pending_admin: Signer<'info>,
}

#[derive(Accounts)]
pub struct AcceptPauserAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.pending_pauser == pending_pauser.
key() @ GatewayError::Unauthorized
    )]
    pub config: Account<'info, Config>,

    pub pending_pauser: Signer<'info>,
}
```

Admin/pauser rotations now use `propose_*` plus explicit acceptance by the pending authority.

3. `UnpauseAction` and `unpause`: moved unpause authority from pauser/admin to operator

Pause and unpause were previously symmetric under the same signer path. The access-control split introduces a dedicated operator role for recovery and TSS maintenance, so unpause now checks `config.operator` instead of `config.pauser`.

Before:

```
#[derive(Accounts)]
pub struct PauseAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.pauser == pauser.key() || config.
admin == pauser.key() @ GatewayError::Unauthorized
```

```
    )]
    pub config: Account<'info, Config>,

    pub pauser: Signer<'info>,
}

pub fn unpause(ctx: Context<PauseAction>) -> Result<()> {
    ctx.accounts.config.paused = false;
    Ok(())
}
```

After:

```
#[derive(Accounts)]
pub struct PauseAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.pauser == pauser.key() || config.
admin == pauser.key() @ GatewayError::Unauthorized
    )]
    pub config: Account<'info, Config>,

    pub pauser: Signer<'info>,
}

#[derive(Accounts)]
pub struct UnpauseAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.operator == operator.key() @ Gate
wayError::Unauthorized
    )]
```

```
    pub config: Account<'info, Config>,

    pub operator: Signer<'info>,
}

pub fn unpause(ctx: Context<UnpauseAction>) -> Result<()> {
    ctx.accounts.config.paused = false;
    Ok(())
}
```

Unpause now requires the operator signer instead of reusing the pauser path.

4. `set_operator` : added a dedicated admin-only operator rotation instruction with zero-address rejection and event emission

The operator role is stored in config and needed its own explicit update path. This change also makes operator rotation observable via a dedicated event.

Before:

```
pub fn set_authorities(
    ctx: Context<SetAuthoritiesAction>,
    new_admin: Option<Pubkey>,
    new_pauser: Option<Pubkey>,
) -> Result<()> {
    // ...
}
```

After:

```
/// Set operator authority (admin-only, available while pause
d).
pub fn set_operator(ctx: Context<AdminAction>, new_operator:
Pubkey) -> Result<()> {
    require!(new_operator != Pubkey::default(), GatewayErro
r::ZeroAddress);
    let old_operator = ctx.accounts.config.operator;
```

```
    ctx.accounts.config.operator = new_operator;
    emit!(crate::state::OperatorChanged { old_operator, new_o
perator });
    Ok(())
}
```

Added a dedicated `set_operator` instruction instead of overloading admin/pauser rotation.

5. `propose_authorities` : pending fields are written instead of replacing live admin/pauser immediately

The old `set_authorities` path updated `config.admin` and `config.pauser` in-place. That is now split into proposal state so the live authority is unchanged until the receiving signer confirms.

Before:

```
pub fn set_authorities(
    ctx: Context<SetAuthoritiesAction>,
    new_admin: Option<Pubkey>,
    new_pauser: Option<Pubkey>,
) -> Result<()> {
    require!(
        new_admin.is_some() || new_pauser.is_some(),
        GatewayError::InvalidInput
    );

    let config = &mut ctx.accounts.config;

    if let Some(next) = new_admin {
        require!(next != Pubkey::default(), GatewayError::Zer
oAddress);
        config.admin = next;
    }

    if let Some(next) = new_pauser {
```

```
        require!(next != Pubkey::default(), GatewayError::Zer
oAddress);
        config.pauser = next;
    }

    Ok(())
}
```

After:

```
pub fn propose_authorities(
    ctx: Context<ProposeAuthoritiesAction>,
    new_admin: Option<Pubkey>,
    new_pauser: Option<Pubkey>,
) -> Result<()> {
    require!(
        new_admin.is_some() || new_pauser.is_some(),
        GatewayError::InvalidInput
    );

    let config = &mut ctx.accounts.config;

    if let Some(next) = new_admin {
        require!(next != Pubkey::default(), GatewayError::Zer
oAddress);
        config.pending_admin = next;
    }

    if let Some(next) = new_pauser {
        require!(next != Pubkey::default(), GatewayError::Zer
oAddress);
        config.pending_pauser = next;
    }
```

```
    Ok(())
}
```

Admin and pauser updates now stage into `pending_admin` and `pending_pauser`.

6. `accept_admin` and `accept_pauser`: new acceptance instructions finalize pending rotations and clear the pending slots

Once proposal state was introduced, the program needed explicit completion instructions. Each acceptance path promotes the pending key into the live role and resets the pending slot to `Pubkey::default()`.

Before:

```
// No accept step existed.
```

After:

```
pub fn accept_admin(ctx: Context<AcceptAdminAction>) -> Resul
t<()> {
    let config = &mut ctx.accounts.config;
    config.admin = ctx.accounts.pending_admin.key();
    config.pending_admin = Pubkey::default();
    Ok(())
}

pub fn accept_pauser(ctx: Context<AcceptPauserAction>) -> Res
ult<()> {
    let config = &mut ctx.accounts.config;
    config.pauser = ctx.accounts.pending_pauser.key();
    config.pending_pauser = Pubkey::default();
    Ok(())
}
```

Added explicit accept instructions for pending admin and pauser rotations.

7. `set_protocol_fee` : added an on-chain upper bound for protocol fee configuration

A flat fee misconfiguration could make inbound deposits unusable or drift far beyond intended operator policy. The setter now enforces the protocol-wide cap defined in state.

Before:

```
pub fn set_protocol_fee(ctx: Context<FeeVaultAdminAction>, fe
e_lamports: u64) -> Result<()> {
    // Keep bump persisted so seeded constraints continue to
validate consistently.
    ctx.accounts.fee_vault.bump = ctx.bumps.fee_vault;
    ctx.accounts.fee_vault.protocol_fee_lamports = fee_lampor
ts;
    emit!(ProtocolFeeUpdated { new_fee_lamports: fee_lamports
});
    Ok(())
}
```

After:

```
pub fn set_protocol_fee(ctx: Context<FeeVaultAdminAction>, fe
e_lamports: u64) -> Result<()> {
    require!(fee_lamports <= MAX_PROTOCOL_FEE_LAMPORTS, Gatew
ayError::InvalidInput);
    // Keep bump persisted so seeded constraints continue to
validate consistently.
    ctx.accounts.fee_vault.bump = ctx.bumps.fee_vault;
    ctx.accounts.fee_vault.protocol_fee_lamports = fee_lampor
ts;
    emit!(ProtocolFeeUpdated {
        new_fee_lamports: fee_lamports
    });
```

```
    Ok(())
}
```

Protocol fee updates now reject values above `MAX_PROTOCOL_FEE_LAMPORTS`.

8. `set_pyth_max_age_seconds`: added an admin setter for inbound Pyth staleness tolerance

Inbound gas-route cap checks previously used a hardcoded Pyth freshness window. This change makes staleness configurable on-chain so deployments can tighten or loosen the acceptance window without a code upgrade.

Before:

```
pub fn set_pyth_confidence_threshold(ctx: Context<AdminAction
>, threshold: u64) -> Result<()> {
    require!(threshold > 0, GatewayError::InvalidAmount);
    ctx.accounts.config.pyth_confidence_threshold = threshol
d;
    Ok(())
}
```

After:

```
pub fn set_pyth_confidence_threshold(ctx: Context<AdminAction
>, threshold: u64) -> Result<()> {
    require!(threshold > 0, GatewayError::InvalidAmount);
    ctx.accounts.config.pyth_confidence_threshold = threshol
d;
    Ok(())
}

pub fn set_pyth_max_age_seconds(ctx: Context<AdminAction>, ma
x_age_seconds: u64) -> Result<()> {
    require!(max_age_seconds > 0, GatewayError::InvalidAmoun
t);
    ctx.accounts.config.pyth_max_age_seconds = max_age_second
```

```
s;
    Ok(())
}
```

Added a dedicated admin setter for `config.pyth_max_age_seconds`.

9. `RateLimitConfigAction`: removed the paused guard from block-cap and epoch-duration setters

Rate limit config is an admin control surface and still needs to work during pauses. Keeping the pause gate here blocked emergency cap changes when the gateway was halted.

Before:

```
#[derive(Accounts)]
pub struct RateLimitConfigAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = !config.paused @ GatewayError::Paused,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,
    // ...
}
```

After:

```
#[derive(Accounts)]
pub struct RateLimitConfigAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
```

```
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,
    // ...
}
```

Rate-limit configuration setters now remain callable while paused.

10. `TokenRateLimitAction`: removed the paused guard from token-level rate-limit administration

Token support and threshold changes are also admin-controlled recovery tools. The pause gate was removed so token settings can still be corrected during an incident.

Before:

```
#[derive(Accounts)]
pub struct TokenRateLimitAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = !config.paused @ GatewayError::Paused,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,
    // ...
}
```

After:

```
#[derive(Accounts)]
pub struct TokenRateLimitAction<'info> {
    #[account(
```

```
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == admin.key() @ GatewayErr
or::Unauthorized
    )]
    pub config: Account<'info, Config>,
    // ...
}
```

Token rate-limit admin instructions are no longer blocked by the paused flag.

11. `set_token_rate_limit` : added mint-authority trust flags, mint validation, and preserved epoch usage across threshold updates

Token rate limits previously accepted any mint account and also reset epoch usage on every threshold update. The revised setter validates SPL mint metadata when enabling support, requires explicit trust acknowledgements for mint/freeze authorities, and preserves current-epoch usage so an admin change cannot wipe consumption state.

Before:

```
/// Set token-specific rate limit threshold (matching EVM set
TokenToLimitThreshold)
/// @param limit_threshold Max amount per epoch (token's natu
ral units). Set to 0 to disable rate limiting for this token.
pub fn set_token_rate_limit(
    ctx: Context<TokenRateLimitAction>,
    limit_threshold: u128,
) -> Result<()> {
    // Allow limit_threshold = 0 to disable rate limiting (ma
tching EVM behavior)
    let token_rate_limit = &mut ctx.accounts.token_rate_limi
t;
    token_rate_limit.token_mint = ctx.accounts.token_mint.key
();
```

```
    token_rate_limit.limit_threshold = limit_threshold;
    token_rate_limit.epoch_usage = EpochUsage { epoch: 0, use
d: 0 };

    // Emit event
    emit!(TokenRateLimitUpdated {
        token_mint: ctx.accounts.token_mint.key(),
        limit_threshold,
    });

    Ok(())
}
```

After:

```
use anchor_spl::token::{Mint, Token};

/// Set token-specific rate limit threshold (matching EVM set
TokenToLimitThreshold)
/// @param limit_threshold Max amount per epoch (token's natu
ral units).
///        Set to 0 to remove support for this token — deposi
ts will be rejected with NotSupported.
///        To disable epoch consumption while keeping the tok
en supported, set epoch_duration_sec to 0.
/// @dev  Epoch usage is intentionally preserved across thres
hold updates (EVM parity).
///       New accounts are zero-initialized by the runtime, s
o no explicit reset is needed on first init.
pub fn set_token_rate_limit(
    ctx: Context<TokenRateLimitAction>,
    limit_threshold: u128,
    trusted_mint_authority: bool,
    trusted_freeze_authority: bool,
) -> Result<()> {
    // limit_threshold == 0 means token is not supported; dep
```

```
osits are rejected with NotSupported.
    let token_mint_key = ctx.accounts.token_mint.key();

    if limit_threshold > 0 && token_mint_key != Pubkey::defau
lt() {
        let token_mint_info = ctx.accounts.token_mint.to_acco
unt_info();
        require!(token_mint_info.owner == &Token::id(), Gatew
ayError::InvalidMint);

        let mint_data = token_mint_info
            .try_borrow_data()
            .map_err(|_| error!(GatewayError::InvalidMint))?;
        let token_mint = Mint::try_deserialize(&mut &mint_dat
a[..])
            .map_err(|_| error!(GatewayError::InvalidMint))?;

        if token_mint.mint_authority.is_some() {
            require!(trusted_mint_authority, GatewayError::In
validMint);
        }

        if token_mint.freeze_authority.is_some() {
            require!(trusted_freeze_authority, GatewayError::
InvalidMint);
        }
    }

    let token_rate_limit = &mut ctx.accounts.token_rate_limi
t;
    token_rate_limit.token_mint = token_mint_key;
    token_rate_limit.limit_threshold = limit_threshold;
    // epoch_usage is NOT reset here — preserving accumulated
usage prevents an admin
    // threshold update from inadvertently clearing the curre
nt-epoch counter (EVM parity).
```

```
    // Emit event
    emit!(TokenRateLimitUpdated {
        token_mint: token_mint_key,
        limit_threshold,
    });

    Ok(())
}
```

Token support configuration now carries explicit mint-trust flags and no longer clears epoch usage on update.

instructions/deposit.rs

1. `send_tx_with_gas_route`: payload-only execution is restricted to `GasAndPayload`, and USD-cap checking now consumes the helper's returned USD amount

The zero-gas path previously accepted `FundsAndPayload` in a branch that should only represent gas-route execution. The pricing helper also started returning computed USD so the deposit flow no longer recomputes price data in a second step.

Before:

```
if gas_amount == 0 {
    require!(
        matches!(tx_type, TxType::GasAndPayload | TxType::Fun
dsAndPayload),
        GatewayError::InvalidAmount
    );

    emit!(UniversalTx {
        sender: ctx.accounts.user.key(),
        recipient: [0u8; 20],
        token: Pubkey::default(),
        amount: 0,
```

```
        payload: payload.to_vec(),
        revert_recipient: *revert_recipient,
        tx_type,
        signature_data: signature_data.to_vec(),
        from_cea: false,
    });

    return Ok(());
}

check_usd_caps(&ctx.accounts.config, gas_amount, &ctx.account
s.price_update)?;
let price_data = calculate_sol_price(&ctx.accounts.price_upda
te)?;
let usd_amount = calculate_usd_amount(gas_amount, &price_dat
a)?;
check_block_usd_cap(&mut ctx.accounts.rate_limit_config, usd_
amount)?;
```

After:

```
if gas_amount == 0 {
    require!(tx_type == TxType::GasAndPayload, GatewayError::
InvalidAmount);

    emit!(UniversalTx {
        sender: ctx.accounts.user.key(),
        recipient: [0u8; 20],
        token: Pubkey::default(),
        amount: 0,
        payload: payload.to_vec(),
        revert_recipient: *revert_recipient,
        tx_type,
        signature_data: signature_data.to_vec(),
        from_cea: false,
    });
```

```
    return Ok(());
}

let usd_amount =
    check_usd_caps(&ctx.accounts.config, gas_amount, &ctx.acc
ounts.price_update)?;
check_block_usd_cap(&mut ctx.accounts.rate_limit_config, usd_
amount)?;
```

Payload-only gas execution is now strictly `GasAndPayload`, and USD-cap checking returns the amount it computed.

2. `send_tx_with_funds_route`: removed redundant payload-shape validation from the funds dispatcher

Route selection already determines whether a request is `Funds` or `FundsAndPayload`. The extra payload-empty/non-empty checks here were redundant and diverged from the routing behavior now used elsewhere.

Before:

```
require!(req.revert_recipient != Pubkey::default(), GatewayEr
ror::InvalidRecipient);
require!(req.amount > 0, GatewayError::InvalidAmount);
if tx_type == TxType::Funds {
    require!(req.payload.is_empty(), GatewayError::InvalidInp
ut);
} else {
    require!(!req.payload.is_empty(), GatewayError::InvalidIn
put);
}
```

After:

```
require!(req.revert_recipient != Pubkey::default(), GatewayEr
ror::InvalidRecipient);
```

```
require!(req.amount > 0, GatewayError::InvalidAmount);
```

Funds-route payload validation now relies on the earlier route derivation logic instead of re-checking here.

3. `deposit_spl_to_vault`: enforced that the destination token account is the vault's canonical ATA

Checking mint and owner alone was not enough, because a user could still point the transfer at a non-canonical vault-owned token account. The function now derives the expected ATA and rejects any other destination account.

Before:

```
let parsed = parse_token_account(&gateway_token_account.to_account_info())?;
require!(parsed.owner == ctx.accounts.vault.key(), GatewayError::InvalidOwner);
require!(parsed.mint == token, GatewayError::InvalidMint);
```

After:

```
use anchor_spl::associated_token::spl_associated_token_account;

let parsed = parse_token_account(&gateway_token_account.to_account_info())?;
require!(parsed.owner == ctx.accounts.vault.key(), GatewayError::InvalidOwner);
require!(parsed.mint == token, GatewayError::InvalidMint);
let expected_gateway_ata =
    spl_associated_token_account::get_associated_token_address(&ctx.accounts.vault.key(), &token);
require!(
    gateway_token_account.key() == expected_gateway_ata,
```

```
        GatewayError::InvalidAccount
    );
```

SPL deposits now only accept the canonical vault ATA as the destination.

instructions/execute.rs

1. `SIGNATURE_FEE_LAMPORTS` and `SPL_TOKEN_ACCOUNT_LEN`: introduced explicit gas-accounting constants for finalize reimbursement

Finalize reimbursement previously paid the full signed `gas_fee` directly to the relayer. The new model needed stable protocol constants to compute actual gas usage from signature cost, `ExecutedSubTx` rent, and optional ATA-creation rent.

Before:

```
use anchor_spl::associated_token::{spl_associated_token_accou
nt, AssociatedToken};
use anchor_spl::token::{spl_token, Mint, Token, TokenAccoun
t};
```

After:

```
use anchor_spl::associated_token::{spl_associated_token_accou
nt, AssociatedToken};
use anchor_spl::token::{spl_token, Mint, Token, TokenAccoun
t};

/// Base Solana transaction fee per signature (protocol const
ant, unchanged since genesis).
///
/// PROTOCOL ASSUMPTION: `caller` is the sole fee payer and t
he finalize transaction has exactly
/// one required signature (the UV/relayer keypair). If the U
V ever uses a separate fee-payer
/// account or a multi-signature setup, `gas_used` will under
```

```
-estimate the actual tx cost and
/// the accounting will drift. This constraint is NOT enforce
d on-chain — it must be upheld
/// by the UV submission service and documented in its operat
ional runbook.
const SIGNATURE_FEE_LAMPORTS: u64 = 5_000;

/// SPL token account data size in bytes (spl_token::state::A
ccount layout — stable protocol constant).
const SPL_TOKEN_ACCOUNT_LEN: usize = 165;
```

Finalize gas accounting now depends on explicit signature-fee and ATA-size constants.

2. `FinalizeUniversalTx` : tightened PDA bump and SPL vault account validation in the account struct

The account struct previously relied on implicit bump handling and only required `token::authority = vault_sol` for the vault ATA. The updated constraints bind to stored bumps and require the provided SPL vault account to be the associated token account for the mint and vault authority.

Before:

```
#[account(
    seeds = [b"config"],
    bump,
)]
pub config: Account<'info, Config>,

#[account(
    mut,
    seeds = [TSS_SEED],
    bump,
)]
pub tss_pda: Account<'info, TssPda>,
```

```
#[account(mut, token::authority = vault_sol)]
pub vault_ata: Option<Account<'info, TokenAccount>>,
```

After:

```
#[account(
    seeds = [b"config"],
    bump = config.bump,
)]
pub config: Account<'info, Config>,

#[account(
    mut,
    seeds = [TSS_SEED],
    bump = tss_pda.bump,
)]
pub tss_pda: Account<'info, TssPda>,

#[account(
    mut,
    associated_token::mint = mint,
    associated_token::authority = vault_sol,
)]
pub vault_ata: Option<Account<'info, TokenAccount>>,
```

Finalize now validates stored bumps explicitly and only accepts the canonical vault ATA for SPL finalization.

3. `finalize_universal_tx`: split staging from gas reimbursement and extended the emitted event payload

Gas reimbursement previously happened inside asset staging and always paid the full signed `gas_fee`. The finalize path now stages assets first, computes actual gas usage, reimburses only that amount, and emits both `gas_used` and `gas_to_refund` plus whether an ATA was created.

Before:

```
stage_assets_to_cea(&ctx, &request, amount, gas_fee, &vault_s
eeds)?;
dispatch_finalize_action(
    &mut ctx,
    &request,
    execute_accounts,
    amount,
    push_account,
    &ix_data,
    &cea_seeds,
)?;

emit!(UniversalTxFinalized {
    sub_tx_id,
    universal_tx_id,
    gas_fee,
    push_account,
    target: request.target,
    token: request.token,
    amount,
    payload: ix_data,
});
```

After:

```
// Stage assets vault → CEA. Returns whether CEA ATA was crea
ted.
let ata_created = stage_assets_to_cea(&ctx, &request, amount,
&vault_seeds)?;

let (gas_used, gas_to_refund) = settle_relayer_gas_cost(&ctx,
gas_fee, ata_created)?;

dispatch_finalize_action(
    &mut ctx,
```

```
        &request,
        execute_accounts,
        amount,
        push_account,
        &ix_data,
        &cea_seeds,
    )?;

    emit!(UniversalTxFinalized {
        sub_tx_id,
        universal_tx_id,
        gas_fee,
        gas_used,
        gas_to_refund,
        ata_created,
        push_account,
        target: request.target,
        token: request.token,
        amount,
        payload: ix_data,
    });
```

Finalize now separates vault staging from gas settlement and emits actual reimbursement data.

4. `settle_relayer_gas_cost`: added a dedicated helper that computes `gas_used`, checks minimum budget, and reimburses only actual cost

The old finalize path transferred `gas_fee` directly to the caller with no model of actual cost or leftover refund. This helper introduces deterministic accounting from rent and signature fee, enforces the minimum budget, and leaves unused gas in the vault for off-chain refund handling.

Before:

```
// No dedicated gas-settlement helper existed.
```

After:

```
#[inline(never)]
fn settle_relayer_gas_cost(
    ctx: &Context<FinalizeUniversalTx>,
    gas_fee: u64,
    ata_created: bool,
) -> Result<(u64, u64)> {
    let sub_tx_rent = Rent::get()?.minimum_balance(ExecutedSu
bTx::LEN);
    let ata_rent = if ata_created {
        Rent::get()?.minimum_balance(SPL_TOKEN_ACCOUNT_LEN)
    } else {
        0
    };
    let gas_used = SIGNATURE_FEE_LAMPORTS + sub_tx_rent + ata
_rent;
    require!(gas_fee >= gas_used, GatewayError::InsufficientG
asBudget);
    let gas_to_refund = gas_fee - gas_used;

    crate::utils::transfer_gas_fee_to_caller(
        &ctx.accounts.vault_sol.to_account_info(),
        &ctx.accounts.caller.to_account_info(),
        &ctx.accounts.system_program.to_account_info(),
        gas_used,
        ctx.accounts.config.vault_bump,
    )?;

    Ok((gas_used, gas_to_refund))
}
```

Added a dedicated gas-settlement helper that reimburses only actual finalize cost.

5. `stage_assets_to_cea` : removed the gas transfer side effect and made the function return `ata_created`

Staging and gas reimbursement were previously conflated, which made it impossible to know whether ATA rent was incurred before paying the relayer. The function now only moves assets and returns the ATA-creation flag needed by later gas settlement.

Before:

```
fn stage_assets_to_cea(
    ctx: &Context<FinalizeUniversalTx>,
    request: &FinalizeRequestContext,
    amount: u64,
    gas_fee: u64,
    vault_seeds: &[&[u8]],
) -> Result<()> {
    if request.is_native {
        pda_system_transfer(
            &ctx.accounts.vault_sol.to_account_info(),
            &ctx.accounts.cea_authority.to_account_info(),
            &ctx.accounts.system_program.to_account_info(),
            amount,
            vault_seeds,
        )?;
    } else {
        process_spl_vault_to_cea_transfer(ctx, amount, vault_
seeds)?;
    }

    crate::utils::transfer_gas_fee_to_caller(
        &ctx.accounts.vault_sol.to_account_info(),
        &ctx.accounts.caller.to_account_info(),
        &ctx.accounts.system_program.to_account_info(),
        gas_fee,
        ctx.accounts.config.vault_bump,
```

```
    )
}
```

After:

```
fn stage_assets_to_cea(
    ctx: &Context<FinalizeUniversalTx>,
    request: &FinalizeRequestContext,
    amount: u64,
    vault_seeds: &[&[u8]],
) -> Result<bool> {
    if request.is_native {
        pda_system_transfer(
            &ctx.accounts.vault_sol.to_account_info(),
            &ctx.accounts.cea_authority.to_account_info(),
            &ctx.accounts.system_program.to_account_info(),
            amount,
            vault_seeds,
        )?;
        Ok(false)
    } else {
        process_spl_vault_to_cea_transfer(ctx, amount, vault_
seeds)
    }
}
```

Asset staging no longer transfers gas and now reports whether a CEA ATA was created.

6. `process_spl_vault_to_cea_transfer` : now reports whether the CEA ATA was created during finalization

Gas reimbursement must include ATA rent only when the finalize path actually created the CEA token account. The SPL staging helper now captures `data_is_empty()` before the ATA CPI, validates the resulting ATA, and returns that boolean.

Before:

```
fn process_spl_vault_to_cea_transfer<'info>(
    ctx: &Context<FinalizeUniversalTx<'info>>,
    amount: u64,
    vault_seeds: &[&[u8]],
) -> Result<()> {
    // ...
    let cea_ata_info = cea_ata.to_account_info();
    if cea_ata_info.data_is_empty() {
        let create_ata_ix =
            spl_associated_token_account::instruction::create
_associated_token_account(
                &ctx.accounts.caller.key(),
                &ctx.accounts.cea_authority.key(),
                &mint.key(),
                &spl_token::ID,
            );
        invoke_signed(
            &create_ata_ix,
            &[
                ctx.accounts.caller.to_account_info(),
                cea_ata.to_account_info(),
                ctx.accounts.cea_authority.to_account_info(),
                mint.to_account_info(),
                ctx.accounts.system_program.to_account_info
(),
                token_program.to_account_info(),
                ata_program.to_account_info(),
                rent.to_account_info(),
            ],
            &[],
        )?;
    }

    // ...
```

```
    pda_spl_transfer(
        &vault_ata.to_account_info(),
        &cea_ata.to_account_info(),
        &ctx.accounts.vault_sol.to_account_info(),
        amount,
        vault_seeds,
    )?;

    Ok(())
}
```

After:

```
fn process_spl_vault_to_cea_transfer<'info>(
    ctx: &Context<FinalizeUniversalTx<'info>>,
    amount: u64,
    vault_seeds: &[&[u8]],
) -> Result<bool> {
    // ...
    let cea_ata_info = cea_ata.to_account_info();
    let ata_created = cea_ata_info.data_is_empty();
    if ata_created {
        let create_ata_ix =
            spl_associated_token_account::instruction::create
_associated_token_account(
                &ctx.accounts.caller.key(),
                &ctx.accounts.cea_authority.key(),
                &mint.key(),
                &spl_token::ID,
            );
        invoke_signed(
            &create_ata_ix,
            &[
                ctx.accounts.caller.to_account_info(),
                cea_ata.to_account_info(),
                ctx.accounts.cea_authority.to_account_info(),
```

```
                mint.to_account_info(),
                ctx.accounts.system_program.to_account_info
(),
                token_program.to_account_info(),
                ata_program.to_account_info(),
                rent.to_account_info(),
            ],
            &[],
        )?;
    }

    // ...
    pda_spl_transfer(
        &vault_ata.to_account_info(),
        &cea_ata.to_account_info(),
        &ctx.accounts.vault_sol.to_account_info(),
        amount,
        vault_seeds,
    )?;

    Ok(ata_created)
}
```

SPL staging now returns `true` only when the finalize path created the CEA ATA.

instructions/initialize.rs

1. `Initialize`: required the program upgrade authority to be the initializing signer

Initialization was previously authorized only by whatever signer paid for the config account, without proving that signer controlled the upgradeable program. The account struct now resolves the program's `ProgramData` and requires its upgrade authority to match `admin`.

Before:

```rust
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(
        init,
        seeds = [CONFIG_SEED],
        bump,
        payer = admin,
        space = Config::LEN
    )]
    pub config: Account<'info, Config>,

    /// CHECK: Native SOL holder, not deserialized
    #[account(
        mut,
        seeds = [VAULT_SEED],
        bump
    )]
    pub vault: UncheckedAccount<'info>,

    #[account(mut)]
    pub admin: Signer<'info>,

    pub system_program: Program<'info, System>,
}
```

After:

```rust
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(
        init,
        seeds = [CONFIG_SEED],
        bump,
        payer = admin,
        space = Config::LEN
```

```
    )]
    pub config: Account<'info, Config>,

    /// CHECK: Native SOL holder, not deserialized
    #[account(
        mut,
        seeds = [VAULT_SEED],
        bump
    )]
    pub vault: UncheckedAccount<'info>,

    pub program: Program<'info, crate::program::UniversalGate
way>,

    #[account(
        constraint = program.programdata_address()? == Some(p
rogram_data.key())
            @ crate::errors::GatewayError::Unauthorized
    )]
    pub program_data: Account<'info, ProgramData>,

    #[account(
        mut,
        constraint = program_data.upgrade_authority_address =
= Some(admin.key())
            @ crate::errors::GatewayError::Unauthorized
    )]
    pub admin: Signer<'info>,

    pub system_program: Program<'info, System>,
}
```

Initialization now proves that the signer is the current upgrade authority for the deployed program.

2. `initialize` : renamed the third authority parameter from `tss` to `operator` and stored it in config

The config layout repurposed the legacy `tss_address` slot into the operator authority. Initialization therefore needed to accept an operator pubkey instead of a dead TSS placeholder.

Before:

```
pub fn initialize(
    ctx: Context<Initialize>,
    admin: Pubkey,
    pauser: Pubkey,
    tss: Pubkey,
    min_cap_usd: u128,
    max_cap_usd: u128,
    pyth_price_feed: Pubkey,
) -> Result<()> {
    // ...
    require!(
        tss != Pubkey::default(),
        crate::errors::GatewayError::ZeroAddress
    );
    // ...
    config.admin = admin;
    config.pauser = pauser;
    config.tss_address = tss;
    // ...

    msg!("Gateway initialized with admin: {}, TSS: {}", admi
n, tss);
    Ok(())
}
```

After:

```rust
pub fn initialize(
    ctx: Context<Initialize>,
    admin: Pubkey,
    pauser: Pubkey,
    operator: Pubkey,
    min_cap_usd: u128,
    max_cap_usd: u128,
    pyth_price_feed: Pubkey,
) -> Result<()> {
    // ...
    require!(
        operator != Pubkey::default(),
        crate::errors::GatewayError::ZeroAddress
    );
    // ...
    config.admin = admin;
    config.pauser = pauser;
    config.operator = operator;
    // ...

    msg!("Gateway initialized with admin: {}, operator: {}",
admin, operator);
    Ok(())
}
```

Initialization now writes an operator authority instead of a legacy TSS placeholder.

3. `initialize`: added zero-address validation for `pauser` and initialized `pyth_max_age_seconds`

The initializer already validated admin, the operator slot, and the Pyth feed, but it did not reject a zero pauser. It also needed to seed the new configurable Pyth freshness window so post-upgrade configs have a non-zero default.

Before:

```
require!(
    admin != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);
require!(
    tss != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);
require!(
    pyth_price_feed != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);

config.pyth_price_feed = pyth_price_feed;
config.pyth_confidence_threshold = 1000000; // Default confid
ence threshold (1e6)
```

After:

```
require!(
    admin != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);
require!(
    operator != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);
require!(
    pauser != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);
require!(
    pyth_price_feed != Pubkey::default(),
    crate::errors::GatewayError::ZeroAddress
);
```

```
config.pyth_price_feed = pyth_price_feed;
config.pyth_confidence_threshold = 1000000; // Default confid
ence threshold (1e6)
config.pyth_max_age_seconds = 60; // Default staleness window
(60 seconds)
```

Initialization now rejects a zero pauser and seeds the new Pyth max-age config with `60` seconds.

instructions/rescue.rs

1. `RescueFunds` : tightened SPL vault account validation to the canonical ATA and removed the manual owner equality check

The rescue path previously accepted any token account owned by the vault PDA. The account struct now enforces that the passed SPL vault is the canonical ATA for `vault` and `token_mint`, making the manual `owner == vault` check redundant.

Before:

```
/// Vault ATA for this mint — holds bridged SPL tokens.
#[account(mut)]
pub token_vault: Option<Account<'info, TokenAccount>>,

// ...

let token_vault = ctx.accounts.token_vault.as_ref().ok_or(err
or!(GatewayError::InvalidAccount))?;
let recipient_ta = ctx.accounts.recipient_token_account.as_re
f().ok_or(error!(GatewayError::InvalidAccount))?;
let mint_key = ctx.accounts.token_mint.as_ref().unwrap().key
();
require!(token_vault.mint == mint_key, GatewayError::InvalidM
int);
require!(token_vault.owner == ctx.accounts.vault.key(), Gatew
ayError::InvalidAccount);
```

```
require!(recipient_ta.mint == mint_key, GatewayError::Invalid
Mint);
require!(recipient_ta.owner == recipient, GatewayError::Inval
idRecipient);
```

After:

```
/// Vault ATA for this mint — holds bridged SPL tokens.
#[account(
    mut,
    associated_token::mint = token_mint,
    associated_token::authority = vault,
)]
pub token_vault: Option<Account<'info, TokenAccount>>,

// ...

let token_vault = ctx.accounts.token_vault.as_ref().ok_or(err
or!(GatewayError::InvalidAccount))?;
let recipient_ta = ctx.accounts.recipient_token_account.as_re
f().ok_or(error!(GatewayError::InvalidAccount))?;
let mint_key = ctx.accounts.token_mint.as_ref().unwrap().key
();
require!(token_vault.mint == mint_key, GatewayError::InvalidM
int);
require!(recipient_ta.mint == mint_key, GatewayError::Invalid
Mint);
require!(recipient_ta.owner == recipient, GatewayError::Inval
idRecipient);
```

Rescue now only accepts the canonical vault ATA for SPL assets.

instructions/revert.rs

1. `RevertUniversalTx` : tightened SPL vault account validation to the canonical ATA and removed the manual owner equality check

The revert path had the same SPL vault-account ambiguity as rescue. The account struct now enforces the vault ATA derivation directly, so the explicit owner equality check is no longer needed.

Before:

```
/// Vault ATA for this mint — holds bridged SPL tokens.
#[account(mut)]
pub token_vault: Option<Account<'info, TokenAccount>>,

// ...

let token_vault = ctx.accounts.token_vault.as_ref().ok_or(err
or!(GatewayError::InvalidAccount))?;
let recipient_ta = ctx.accounts.recipient_token_account.as_re
f().ok_or(error!(GatewayError::InvalidAccount))?;
let mint_key = ctx.accounts.token_mint.as_ref().unwrap().key
();
require!(token_vault.mint == mint_key, GatewayError::InvalidM
int);
require!(token_vault.owner == ctx.accounts.vault.key(), Gatew
ayError::InvalidAccount);
require!(recipient_ta.mint == mint_key, GatewayError::Invalid
Mint);
require!(recipient_ta.owner == recipient, GatewayError::Inval
idRecipient);
```

After:

```
/// Vault ATA for this mint — holds bridged SPL tokens.
#[account(
    mut,
    associated_token::mint = token_mint,
    associated_token::authority = vault,
```

```
)]
pub token_vault: Option<Account<'info, TokenAccount>>,

// ...

let token_vault = ctx.accounts.token_vault.as_ref().ok_or(err
or!(GatewayError::InvalidAccount))?;
let recipient_ta = ctx.accounts.recipient_token_account.as_re
f().ok_or(error!(GatewayError::InvalidAccount))?;
let mint_key = ctx.accounts.token_mint.as_ref().unwrap().key
();
require!(token_vault.mint == mint_key, GatewayError::InvalidM
int);
require!(recipient_ta.mint == mint_key, GatewayError::Invalid
Mint);
require!(recipient_ta.owner == recipient, GatewayError::Inval
idRecipient);
```

Revert now only accepts the canonical vault ATA for SPL assets.

2. `revert_universal_tx` : added `revert_msg_hash` to the TSS-authenticated message format

The old TSS message authenticated recipient, token, amount, and gas fee, but it did not bind the actual revert payload bytes. The revert path now hashes `revert_instruction.revert_msg` on-chain and includes that hash in the signed message so relayers cannot swap revert payloads under an otherwise valid signature.

Before:

```
// TSS message format (instruction_id = 3 for both modes):
//   SOL: amount || [sub_tx_id, universal_tx_id, recipient, g
as_fee]
//   SPL: amount || [sub_tx_id, universal_tx_id, mint, recipi
ent, gas_fee]

let recipient = ctx.accounts.recipient.key();
```

```
require!(revert_instruction.revert_recipient != Pubkey::default(), GatewayError::InvalidRecipient);
require!(recipient == revert_instruction.revert_recipient, GatewayError::InvalidRecipient);

// ...

// TSS message: instruction_id=3 || amount || [sub_tx_id, universal_tx_id, (mint,) recipient, gas_fee]
let recipient_bytes = recipient.to_bytes();
let gas_fee_buf = encode_u64_be(gas_fee);
if is_native {
    let additional: [&[u8]; 4] = [&sub_tx_id, &universal_tx_id, &recipient_bytes, &gas_fee_buf];
    validate_message(&mut ctx.accounts.tss_pda, 3, Some(amount), &additional, &message_hash, &signature, recovery_id)?;
} else {
    let mint_bytes = ctx.accounts.token_mint.as_ref().unwrap().key().to_bytes();
    let additional: [&[u8]; 5] = [&sub_tx_id, &universal_tx_id, &mint_bytes, &recipient_bytes, &gas_fee_buf];
    validate_message(&mut ctx.accounts.tss_pda, 3, Some(amount), &additional, &message_hash, &signature, recovery_id)?;
}
```

After:

```
use anchor_lang::solana_program::keccak::hash;

// TSS message format (instruction_id = 3 for both modes):
//   SOL: amount || [sub_tx_id, universal_tx_id, recipient, gas_fee, revert_msg_hash]
//   SPL: amount || [sub_tx_id, universal_tx_id, mint, recipient, gas_fee, revert_msg_hash]

let recipient = ctx.accounts.recipient.key();
```

```
require!(revert_instruction.revert_recipient != Pubkey::default(), GatewayError::InvalidRecipient);
require!(recipient == revert_instruction.revert_recipient, GatewayError::InvalidRecipient);
let revert_msg_hash = hash(revert_instruction.revert_msg.as_slice()).to_bytes();

// ...

// TSS message: instruction_id=3 || amount || [sub_tx_id, universal_tx_id, (mint,) recipient, gas_fee, revert_msg_hash]
let recipient_bytes = recipient.to_bytes();
let gas_fee_buf = encode_u64_be(gas_fee);
if is_native {
    let additional: [&[u8]; 5] = [&sub_tx_id, &universal_tx_id, &recipient_bytes, &gas_fee_buf, &revert_msg_hash];
    validate_message(&mut ctx.accounts.tss_pda, 3, Some(amount), &additional, &message_hash, &signature, recovery_id)?;
} else {
    let mint_bytes = ctx.accounts.token_mint.as_ref().unwrap().key().to_bytes();
    let additional: [&[u8]; 6] = [&sub_tx_id, &universal_tx_id, &mint_bytes, &recipient_bytes, &gas_fee_buf, &revert_msg_hash];
    validate_message(&mut ctx.accounts.tss_pda, 3, Some(amount), &additional, &message_hash, &signature, recovery_id)?;
}
```

Revert signatures now bind the revert-message bytes via `keccak(revert_msg)`.

instructions/tss.rs

1. `init_tss`: stopped writing the legacy `authority` field during initialization

The `authority` field remains in `TssPda` only for account-layout compatibility. Since it no longer drives authorization, `init_tss` no longer writes an effectively dead admin/operator snapshot into that slot.

Before:

```
pub fn init_tss(ctx: Context<InitTss>, tss_eth_address: [u8;
20], chain_id: String) -> Result<()> {
    let tss = &mut ctx.accounts.tss_pda;
    tss.tss_eth_address = tss_eth_address;
    tss.chain_id = chain_id;
    tss.authority = ctx.accounts.authority.key();
    tss.bump = ctx.bumps.tss_pda;
    Ok(())
}
```

After:

```
pub fn init_tss(ctx: Context<InitTss>, tss_eth_address: [u8;
20], chain_id: String) -> Result<()> {
    let tss = &mut ctx.accounts.tss_pda;
    tss.tss_eth_address = tss_eth_address;
    tss.chain_id = chain_id;
    tss.bump = ctx.bumps.tss_pda;
    Ok(())
}
```

`init_tss` now leaves the legacy `authority` field untouched instead of writing a no-longer-used signer.

2. `UpdateTss`: changed TSS maintenance authority from admin to operator

TSS rotation is operational maintenance, not long-term governance. The access-control split therefore moved `update_tss` from the admin key to the new operator authority.

Before:

```rust
/// Update TSS ETH address / chain id (admin-only)
#[derive(Accounts)]
pub struct UpdateTss<'info> {
    #[account(
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == authority.key() @ Gatewa
yError::Unauthorized
    )]
    pub config: Account<'info, Config>,
    // ...
}
```

After:

```rust
/// Update TSS ETH address / chain id (operator-only)
#[derive(Accounts)]
pub struct UpdateTss<'info> {
    #[account(
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.operator == authority.key() @ Gat
ewayError::Unauthorized
    )]
    pub config: Account<'info, Config>,
    // ...
}
```

TSS rotation now requires the operator signer instead of the admin signer.

instructions/withdraw.rs

1. `send_universal_tx_to_uea` : allowed payload-only self-calls by requiring “amount or payload” instead of “amount only”

The CEA-to-UEA helper previously rejected any self-call with `amount == 0`, which blocked payload-only executions. The validation now allows zero-amount requests as long as the payload is non-empty.

Before:

```
require!(args.token == token, GatewayError::InvalidMint);
require!(args.amount > 0, GatewayError::InvalidAmount);
require!(args.revert_recipient != Pubkey::default(), GatewayE
rror::InvalidRecipient);
```

After:

```
require!(args.token == token, GatewayError::InvalidMint);
// At least one of amount or payload must be present
require!(
    args.amount > 0 || !args.payload.is_empty(),
    GatewayError::InvalidInput
);
require!(
    args.revert_recipient != Pubkey::default(),
    GatewayError::InvalidRecipient
);
```

CEA self-calls can now be payload-only as long as they still include a valid revert recipient.

2. `send_universal_tx_to_uea`: only consumes rate limit and transfers funds when `withdraw_amount > 0`

The old implementation always performed balance checks, rate-limit consumption, and transfer logic, even for payload-only calls. That made zero-amount self-calls impossible and incorrectly tied payload execution to funds movement.

Before:

```
let rl_config = ctx
    .accounts
    .rate_limit_config
    .as_ref()
    .ok_or(error!(GatewayError::InvalidAccount))?;
let token_rate_limit = ctx
    .accounts
    .token_rate_limit
    .as_mut()
    .ok_or(error!(GatewayError::InvalidAccount))?;

validate_token_and_consume_rate_limit(
    token_rate_limit,
    token,
    withdraw_amount as u128,
    rl_config,
)?;

if args.token == Pubkey::default() {
    pda_system_transfer(
        &ctx.accounts.cea_authority.to_account_info(),
        &ctx.accounts.vault_sol.to_account_info(),
        &ctx.accounts.system_program.to_account_info(),
        withdraw_amount,
        cea_seeds,
    )?;
} else {
    let vault_ata = ctx
        .accounts
        .vault_ata
        .as_ref()
        .ok_or(error!(GatewayError::InvalidAccount))?;
    let cea_ata = ctx
        .accounts
        .cea_ata
```

```
            .as_ref()
            .ok_or(error!(GatewayError::InvalidAccount))?;

        pda_spl_transfer(
            &cea_ata.to_account_info(),
            &vault_ata.to_account_info(),
            &ctx.accounts.cea_authority.to_account_info(),
            withdraw_amount,
            cea_seeds,
        )?;
    }
```

After:

```
    if withdraw_amount > 0 {
        let rl_config = ctx
            .accounts
            .rate_limit_config
            .as_ref()
            .ok_or(error!(GatewayError::InvalidAccount))?;
        let token_rate_limit = ctx
            .accounts
            .token_rate_limit
            .as_mut()
            .ok_or(error!(GatewayError::InvalidAccount))?;

        validate_token_and_consume_rate_limit(
            token_rate_limit,
            token,
            withdraw_amount as u128,
            rl_config,
        )?;

        if token == Pubkey::default() {
            require!(
                withdraw_amount <= ctx.accounts.cea_authority.lam
```

```
ports(),
            GatewayError::InsufficientBalance
        );
        pda_system_transfer(
            &ctx.accounts.cea_authority.to_account_info(),
            &ctx.accounts.vault_sol.to_account_info(),
            &ctx.accounts.system_program.to_account_info(),
            withdraw_amount,
            cea_seeds,
        )?;
    } else {
        let cea_ata = ctx
            .accounts
            .cea_ata
            .as_ref()
            .ok_or(error!(GatewayError::InvalidAccount))?;
        let vault_ata = ctx
            .accounts
            .vault_ata
            .as_ref()
            .ok_or(error!(GatewayError::InvalidAccount))?;
        let parsed_cea_ata = parse_token_account(&cea_ata.to_
account_info())?;
        require!(
            withdraw_amount <= parsed_cea_ata.amount,
            GatewayError::InsufficientBalance
        );
        pda_spl_transfer(
            &cea_ata.to_account_info(),
            &vault_ata.to_account_info(),
            &ctx.accounts.cea_authority.to_account_info(),
            withdraw_amount,
            cea_seeds,
        )?;
```

```
    }
}
```

Zero-amount self-calls now skip balance checks, rate limiting, and vault replenishment transfers.

3. `send_universal_tx_to_uea`: emitted `GasAndPayload` for payload-only self-calls

Once payload-only self-calls became valid, the emitted transaction type also had to reflect that they are execution-only. The helper now derives `GasAndPayload` when `withdraw_amount == 0`.

Before:

```
let tx_type = if args.payload.is_empty() {
    TxType::Funds
} else {
    TxType::FundsAndPayload
};
```

After:

```
let tx_type = match (withdraw_amount > 0, args.payload.is_emp
ty()) {
    (true, true) => TxType::Funds,
    (true, false) => TxType::FundsAndPayload,
    (false, _) => TxType::GasAndPayload, // payload-only, no
funds transferred
};
```

Payload-only self-calls now emit `TxType::GasAndPayload` instead of pretending they moved funds.

lib.rs

1. `initialize` : changed the public instruction signature from `tss` to `operator`

The operator role replaced the old unused config slot. The public program entrypoint now exposes that role directly so SDKs and deployment scripts must pass an operator pubkey at initialization time.

Before:

```
pub fn initialize(
    ctx: Context<Initialize>,
    admin: Pubkey,
    pauser: Pubkey,
    tss: Pubkey,
    min_cap_usd: u128,
    max_cap_usd: u128,
    pyth_price_feed: Pubkey,
) -> Result<()> {
    instructions::initialize::initialize(
        ctx,
        admin,
        pauser,
        tss,
        min_cap_usd,
        max_cap_usd,
        pyth_price_feed,
    )
}
```

After:

```
pub fn initialize(
    ctx: Context<Initialize>,
    admin: Pubkey,
    pauser: Pubkey,
    operator: Pubkey,
    min_cap_usd: u128,
```

```
    max_cap_usd: u128,
    pyth_price_feed: Pubkey,
) -> Result<()> {
    instructions::initialize::initialize(
        ctx,
        admin,
        pauser,
        operator,
        min_cap_usd,
        max_cap_usd,
        pyth_price_feed,
    )
}
```

Program initialization now takes an operator pubkey instead of a legacy TSS placeholder.

2. `pause` / `unpause` / authority management entrypoints: exposed the new operator split and the two-step authority flow

The public program interface needed to match the new on-chain authority model. This added `set_operator`, renamed `set_authorities` to `propose_authorities`, and exposed the new accept instructions while also switching `unpause` to its operator-specific account context.

Before:

```
pub fn pause(ctx: Context<PauseAction>) -> Result<()> {
    instructions::admin::pause(ctx)
}

pub fn unpause(ctx: Context<PauseAction>) -> Result<()> {
    instructions::admin::unpause(ctx)
}

pub fn set_authorities(
    ctx: Context<SetAuthoritiesAction>,
```

```
    new_admin: Option<Pubkey>,
    new_pauser: Option<Pubkey>,
) -> Result<()> {
    instructions::admin::set_authorities(ctx, new_admin, new_
pauser)
}
```

After:

```
pub fn pause(ctx: Context<PauseAction>) -> Result<()> {
    instructions::admin::pause(ctx)
}

pub fn unpause(ctx: Context<UnpauseAction>) -> Result<()> {
    instructions::admin::unpause(ctx)
}

pub fn set_operator(ctx: Context<AdminAction>, new_operator:
Pubkey) -> Result<()> {
    instructions::admin::set_operator(ctx, new_operator)
}

pub fn propose_authorities(
    ctx: Context<ProposeAuthoritiesAction>,
    new_admin: Option<Pubkey>,
    new_pauser: Option<Pubkey>,
) -> Result<()> {
    instructions::admin::propose_authorities(ctx, new_admin,
new_pauser)
}

pub fn accept_admin(ctx: Context<AcceptAdminAction>) -> Resul
t<()> {
    instructions::admin::accept_admin(ctx)
}
```

```
pub fn accept_pauser(ctx: Context<AcceptPauserAction>) -> Res
ult<()> {
    instructions::admin::accept_pauser(ctx)
}
```

The public admin surface now includes operator rotation and pending-authority acceptance.

3. `set_protocol_fee` and `set_pyth_max_age_seconds`: expanded the public admin interface for fee caps and Pyth staleness control

The fee setter now carries an explicit contract-level cap in its interface docs, and a brand-new entrypoint was added for Pyth max-age configuration. Integrators that mirror the admin surface need to expose both.

Before:

```
/// @notice Set flat protocol fee (lamports) for inbound send
_universal_tx.
/// Not gated by `!config.paused` so the admin can disable fe
es during an emergency pause.
pub fn set_protocol_fee(ctx: Context<FeeVaultAdminAction>, fe
e_lamports: u64) -> Result<()> {
    instructions::admin::set_protocol_fee(ctx, fee_lamports)
}

/// @notice Set Pyth confidence threshold
pub fn set_pyth_confidence_threshold(ctx: Context<AdminAction
>, threshold: u64) -> Result<()> {
    instructions::admin::set_pyth_confidence_threshold(ctx, t
hreshold)
}
```

After:

```
/// @notice Set flat protocol fee (lamports) for inbound send
_universal_tx.
```

```
/// Must be <= `MAX_PROTOCOL_FEE_LAMPORTS`.
/// Not gated by `!config.paused` so the admin can disable fe
es during an emergency pause.
pub fn set_protocol_fee(ctx: Context<FeeVaultAdminAction>, fe
e_lamports: u64) -> Result<()> {
    instructions::admin::set_protocol_fee(ctx, fee_lamports)
}

/// @notice Set Pyth confidence threshold
pub fn set_pyth_confidence_threshold(ctx: Context<AdminAction
>, threshold: u64) -> Result<()> {
    instructions::admin::set_pyth_confidence_threshold(ctx, t
hreshold)
}

/// @notice Set Pyth price staleness window (seconds). Applie
s to inbound gas-route cap enforcement.
pub fn set_pyth_max_age_seconds(ctx: Context<AdminAction>, ma
x_age_seconds: u64) -> Result<()> {
    instructions::admin::set_pyth_max_age_seconds(ctx, max_ag
e_seconds)
}
```

The public admin API now includes a dedicated setter for Pyth max-age and documents the protocol-fee cap.

4. `set_token_rate_limit`: added the two mint-trust booleans to the public instruction signature

The underlying admin setter now validates mint and freeze authorities, so the program entrypoint had to forward the new trust acknowledgements. Any SDK or scripting layer calling this instruction must now provide both booleans.

Before:

```
pub fn set_token_rate_limit(
    ctx: Context<TokenRateLimitAction>,
```

```
    limit_threshold: u128,
) -> Result<()> {
    instructions::admin::set_token_rate_limit(ctx, limit_thre
shold)
}
```

After:

```
pub fn set_token_rate_limit(
    ctx: Context<TokenRateLimitAction>,
    limit_threshold: u128,
    trusted_mint_authority: bool,
    trusted_freeze_authority: bool,
) -> Result<()> {
    instructions::admin::set_token_rate_limit(
        ctx,
        limit_threshold,
        trusted_mint_authority,
        trusted_freeze_authority,
    )
}
```

Token rate-limit configuration now requires two additional boolean arguments.

5. `pub use instructions::admin`: updated re-exports to the new authority-management types

The crate-level exports changed to reflect the renamed and newly added admin instruction account types. Downstream Rust consumers importing these account contexts from the crate root need to switch to the new names.

Before:

```
pub use instructions::admin::{
    AdminAction, FeeVaultAdminAction, PauseAction, RateLimitC
```

```
onfigAction, SetAuthoritiesAction, TokenRateLimitAction,
};
```

After:

```
pub use instructions::admin::{
    AdminAction, FeeVaultAdminAction, PauseAction, ProposeAut
horitiesAction, RateLimitConfigAction,
    TokenRateLimitAction,
};
```

The old `SetAuthoritiesAction` re-export was replaced with `ProposeAuthoritiesAction`.

state.rs

1. `MAX_PROTOCOL_FEE_LAMPORTS` : added a protocol-wide upper bound constant for inbound fee configuration

The fee setter now enforces a maximum lamport fee. This constant is the on-chain source of truth for that bound.

Before:

```
pub const EXECUTED_SUB_TX_SEED: &[u8] = b"executed_sub_tx";
pub const CEA_SEED: &[u8] = b"push_identity";
```

After:

```
pub const EXECUTED_SUB_TX_SEED: &[u8] = b"executed_sub_tx";
pub const CEA_SEED: &[u8] = b"push_identity";
pub const MAX_PROTOCOL_FEE_LAMPORTS: u64 = 2_000_000;
```

Added `MAX_PROTOCOL_FEE_LAMPORTS` as the on-chain cap for protocol-fee updates.

2. `Config` : repurposed the legacy `tss_address` slot into `operator` , added pending authority fields, added `pyth_max_age_seconds` , and resized the account

The config account layout was extended without migration by reusing the old unused `tss_address` slot as the live operator field. It also gained pending admin/pauser slots and a configurable Pyth staleness value, so `Config::LEN` had to be recalculated.

Before:

```
#[account]
pub struct Config {
    pub admin: Pubkey,
    /// Legacy field — reserved for account layout compatibil
ity.
    /// Cannot be removed without a migration because it is p
art of the deployed on-chain layout.
    pub tss_address: Pubkey,
    pub pauser: Pubkey,
    pub min_cap_universal_tx_usd: u128,
    pub max_cap_universal_tx_usd: u128,
    pub paused: bool,
    pub bump: u8,
    pub vault_bump: u8,
    pub pyth_price_feed: Pubkey,
    pub pyth_confidence_threshold: u64,
}

impl Config {
    // discriminator + fields + padding
    // 8 + 32 + 32 + 32 + 16 + 16 + 1 + 1 + 1 + 32 + 8 + 100
    pub const LEN: usize = 8 + 32 + 32 + 32 + 16 + 16 + 1 + 1
+ 1 + 32 + 8 + 100;
}
```

After:

```
#[account]
pub struct Config {
```

```
    pub admin: Pubkey,
    /// Operator authority (reusing legacy slot to preserve d
eployed account layout).
    /// This field previously held an unused legacy value (`t
ss_address`).
    pub operator: Pubkey,
    pub pauser: Pubkey,
    pub min_cap_universal_tx_usd: u128,
    pub max_cap_universal_tx_usd: u128,
    pub paused: bool,
    pub bump: u8,
    pub vault_bump: u8,
    pub pyth_price_feed: Pubkey,
    pub pyth_confidence_threshold: u64,
    pub pending_admin: Pubkey,
    pub pending_pauser: Pubkey,
    /// Maximum allowed age for Pyth price updates (seconds).
Admin-configurable.
    /// Default: 60. Applies to inbound gas-route USD cap enf
orcement only.
    pub pyth_max_age_seconds: u64,
}

impl Config {
    // discriminator + fields + padding
    // 8 + 32 + 32 + 32 + 16 + 16 + 1 + 1 + 1 + 32 + 8 + 32 +
32 + 8 + 28
    pub const LEN: usize = 8 + 32 + 32 + 32 + 16 + 16 + 1 + 1
+ 1 + 32 + 8 + 32 + 32 + 8 + 28;
}
```

`Config` now stores operator and pending authority state plus configurable Pyth max-age.

3. `TssPda`: clarified the legacy `authority` field semantics after operator split

The TSS account layout still includes `authority`, but that field is now explicitly documented as a zero-initialized legacy slot. This matters for integrators reading raw account state directly.

Before:

```
#[account]
pub struct TssPda {
    pub tss_eth_address: [u8; 20],
    pub chain_id: String,
    /// Legacy field — set at init_tss but no longer used for
authorization.
    /// update_tss now checks config.admin. Kept for account
layout compatibility.
    pub authority: Pubkey,
    pub bump: u8,
}
```

After:

```
#[account]
pub struct TssPda {
    pub tss_eth_address: [u8; 20],
    pub chain_id: String,
    /// Legacy field — zero-initialized and no longer used fo
r authorization.
    /// update_tss now checks config.operator. Kept for accou
nt layout compatibility.
    pub authority: Pubkey,
    pub bump: u8,
}
```

The TSS account still contains `authority`, but it is now explicitly legacy and no longer reflects active authorization.

4. `UniversalTxFinalized` : expanded the event with actual gas settlement fields

Finalize events used to expose only the signed `gas_fee`. Off-chain refund and accounting logic now needs to know the actual reimbursement, the user refund remainder, and whether ATA creation was paid for.

Before:

```
#[event]
pub struct UniversalTxFinalized {
    pub sub_tx_id: [u8; 32],
    pub universal_tx_id: [u8; 32],
    pub gas_fee: u64,
    pub push_account: [u8; 20],
    pub target: Pubkey,
    pub token: Pubkey,
    pub amount: u64,
    pub payload: Vec<u8>,
}
```

After:

```
#[event]
pub struct UniversalTxFinalized {
    pub sub_tx_id: [u8; 32],
    pub universal_tx_id: [u8; 32], // Universal transaction I
D from source chain
    pub gas_fee: u64,              // Signed gas budget (lamp
orts) — what TSS authorized
    pub gas_used: u64,             // Actual relayer reimburs
ement (lamports)
    pub gas_to_refund: u64,        // Unused gas returned to
user on Push Chain
    pub ata_created: bool,         // Whether CEA ATA was cre
ated in this finalize
    pub push_account: [u8; 20],
```

```
    pub target: Pubkey,
    pub token: Pubkey,
    pub amount: u64,
    pub payload: Vec<u8>,
}
```

Finalize events now distinguish signed gas budget from actual relayer reimbursement and user refund.

5. `OperatorChanged` : added a new event for operator rotations

Operator authority can now change independently of admin/pauser, so rotations needed their own observable event. This gives SDK and infra consumers a direct log signal instead of forcing config-account polling.

Before:

```
#[event]
pub struct RevertUniversalTx {
    pub sub_tx_id: [u8; 32],
    pub universal_tx_id: [u8; 32],
    pub revert_recipient: Pubkey,
    pub token: Pubkey,
    pub amount: u64,
    pub revert_instruction: RevertInstructions,
}
```

After:

```
#[event]
pub struct RevertUniversalTx {
    pub sub_tx_id: [u8; 32],
    pub universal_tx_id: [u8; 32],
    pub revert_recipient: Pubkey,
    pub token: Pubkey,
    pub amount: u64,
    pub revert_instruction: RevertInstructions,
```

```
}

#[event]
pub struct OperatorChanged {
    pub old_operator: Pubkey,
    pub new_operator: Pubkey,
}
```

Added a dedicated `OperatorChanged` event for operator rotations.

utils/pricing.rs

1. `calculate_sol_price` : replaced the hardcoded Pyth freshness constant with a caller-provided `max_age_seconds`

Price freshness was previously hardcoded to one hour for every path using the helper. The helper now accepts a max-age parameter so inbound cap enforcement can use a stricter configurable window.

Before:

```
/// Maximum allowed age for Pyth price updates used by inboun
d USD-cap checks.
/// Tune before mainnet if tighter freshness is required.
const MAX_PRICE_AGE_SECONDS: u64 = 3_600; // 1 hour

pub fn calculate_sol_price(price_update: &Account<PriceUpdate
V2>) -> Result<PriceData> {
    let feed_id = get_feed_id_from_hex(FEED_ID).map_err(|_| e
rror!(GatewayError::InvalidPrice))?;
    let clock = Clock::get()?;
    let price = price_update
        .get_price_no_older_than(&clock, MAX_PRICE_AGE_SECOND
S, &feed_id)
        .map_err(|_| error!(GatewayError::InvalidPrice))?;
```

After:

```
pub fn calculate_sol_price(
    price_update: &Account<PriceUpdateV2>,
    max_age_seconds: u64,
) -> Result<PriceData> {
    let feed_id = get_feed_id_from_hex(FEED_ID).map_err(|_| e
rror!(GatewayError::InvalidPrice))?;
    let clock = Clock::get()?;
    let price = price_update
        .get_price_no_older_than(&clock, max_age_seconds, &fe
ed_id)
        .map_err(|_| error!(GatewayError::InvalidPrice))?;
```

Price freshness is now controlled by the caller instead of a fixed module constant.

2. `check_usd_caps` : returns computed USD amount and uses `config.pyth_max_age_seconds` with a zero-value fallback

Callers previously had to recompute USD amount after cap validation, and cap enforcement always used the same fixed staleness window. The helper now returns the computed USD amount and uses the config value, defaulting to `60` seconds for legacy configs whose new field is still zero.

Before:

```
pub fn check_usd_caps(
    config: &Config,
    lamports: u64,
    price_update: &Account<PriceUpdateV2>,
) -> Result<()> {
    let price_data = calculate_sol_price(price_update)?;
    if config.pyth_confidence_threshold > 0 {
        require!(
            price_data.confidence <= config.pyth_confidence_t
hreshold,
            GatewayError::InvalidPrice
        );
```

```
    }
    let usd_amount = calculate_usd_amount(lamports, &price_da
ta)?;
    require!(
        usd_amount >= config.min_cap_universal_tx_usd,
        GatewayError::BelowMinCap
    );
    require!(
        usd_amount <= config.max_cap_universal_tx_usd,
        GatewayError::AboveMaxCap
    );
    Ok(())
}
```

After:

```
pub fn check_usd_caps(
    config: &Config,
    lamports: u64,
    price_update: &Account<PriceUpdateV2>,
) -> Result<u128> {
    // Fallback to 60 s when the stored value is 0 (existing
accounts before the field was added).
    let max_age = if config.pyth_max_age_seconds == 0 { 60 }
else { config.pyth_max_age_seconds };
    let price_data = calculate_sol_price(price_update, max_ag
e)?;
    if config.pyth_confidence_threshold > 0 {
        require!(
            price_data.confidence <= config.pyth_confidence_t
hreshold,
            GatewayError::InvalidPrice
        );
    }
    let usd_amount = calculate_usd_amount(lamports, &price_da
ta)?;
```

```
    require!(
        usd_amount >= config.min_cap_universal_tx_usd,
        GatewayError::BelowMinCap
    );
    require!(
        usd_amount <= config.max_cap_universal_tx_usd,
        GatewayError::AboveMaxCap
    );
    Ok(usd_amount)
}
```

USD-cap checking now returns the amount it validated and uses config-driven Pyth freshness.

3. `get_sol_price`: pinned the display-only view path to a one-hour window explicitly

Once `calculate_sol_price` became parameterized, the public view helper needed to choose its own policy. The display-only path now explicitly keeps the older one-hour window because it does not gate protocol behavior.

Before:

```
/// View function for SOL price (locker-compatible)
/// Anyone can fetch SOL price in USD
/// This is the core utility function - the Anchor account st
ruct wrapper is in instructions/price.rs
pub fn get_sol_price(price_update: &Account<PriceUpdateV2>) -
> Result<PriceData> {
    calculate_sol_price(price_update)
}
```

After:

```
/// View function for SOL price (display only — not part of c
ap enforcement).
/// Uses a 1-hour window since this path does not gate any pr
```

```
otocol limit.
pub fn get_sol_price(price_update: &Account<PriceUpdateV2>) -
> Result<PriceData> {
    calculate_sol_price(price_update, 3_600)
}
```

The public SOL-price view now explicitly uses a one-hour staleness window.

Summary of Breaking Changes for Integrators

Instruction signature changes

- `initialize(ctx, admin, pauser, tss, min_cap_usd, max_cap_usd, pyth_price_feed)` → `initialize(ctx, admin, pauser, operator, min_cap_usd, max_cap_usd, pyth_price_feed)`
  - account context also changed: `Initialize` now requires `program` and `program_data`, and the signer must be the program upgrade authority
- `unpause` now uses `Context<UnpauseAction>` instead of `Context<PauseAction>`
  - account context changed from `pauser: Signer` to `operator: Signer`
- `set_authorities(ctx, new_admin, new_pauser)` was removed and replaced by:
  - `propose_authorities(ctx, new_admin, new_pauser)`
  - `accept_admin(ctx)`
  - `accept_pauser(ctx)`
- New instruction: `set_operator(ctx, new_operator)`
- New instruction: `set_pyth_max_age_seconds(ctx, max_age_seconds)`
- `set_token_rate_limit(ctx, limit_threshold)` → `set_token_rate_limit(ctx, limit_threshold, trusted_mint_authority, trusted_freeze_authority)`
- `update_tss` keeps the same arguments, but its required authority changed from `config.admin` to `config.operator`

Account struct changes

- `Config`
  - `tss_address` was repurposed to `operator` in the same storage slot

    - added `pending_admin: Pubkey`
    - added `pending_pauser: Pubkey`
    - added `pyth_max_age_seconds: u64`
    - `Config::LEN` changed
- `UniversalTxFinalized` event payload
    - existing `gas_fee` semantics changed from “full relayer reimbursement” to “signed gas budget”
    - added `gas_used: u64`
    - added `gas_to_refund: u64`
    - added `ata_created: bool`
- `TssPda.authority`
    - still exists in layout
    - now explicitly legacy / no longer used for authorization

PDA seed changes

- No PDA seed strings changed.
- Several account constraints became stricter even without seed changes:
    - finalize/revert/rescue SPL vault accounts now must be canonical associated token accounts where applicable
    - SPL deposit destination must be the vault’s canonical ATA

New instructions added

- `set_operator`
- `accept_admin`
- `accept_pauser`
- `set_pyth_max_age_seconds`

New error codes added

- `InsufficientGasBudget`

Event/log format changes

- `UniversalTxFinalized` now emits:
  - `gas_fee` as signed gas budget
  - `gas_used`
  - `gas_to_refund`
  - `ata_created`
- New event:
  - `OperatorChanged { old_operator, new_operator }`